

Adobe Tags

Third Party Pixel Implementation Playbook

Implementation Guide for Citi Marketing and Analytics Teams

Configure, maintain and test marketing pixels using clear Launch events, conditions and actions. Use the BridgeTrack implementation as the first vendor example, and follow the common process for future requirements.

Document information	Details
Audience	Citi P1T/media team, Adobe Tags implementation engineers and Web Analytics developers
Purpose	Practical configuration and maintenance of third-party marketing and conversion tags
Document owner	[Add team or document owner]
Revision date	8 October 2026

How to use this playbook

Start with Part 1 to understand the common rule setup. Follow Parts A to E in Part 2 for BridgeTrack. Use Part 3 for troubleshooting, new vendor requirements and implementation records.

Part	What it covers
1 General Adobe Tags pixel implementation	Terms, rule behaviour and preparation
2 Vendor 1 BridgeTrack	Existing migration, App Start, application events, Data Elements and testing
3 Common support and future implementation	Troubleshooting, new vendor steps and documentation

Part 1 General Adobe Tags Pixel Implementation

1 Purpose and audience

This playbook helps the Citi P1T/media and implementation teams understand when a pixel should run, which values it needs and how to configure the Launch rule. It also provides a common process for adding new vendor requirements.

2 Third party pixels and Adobe Tags

A third-party pixel or tag sends relevant page, interaction or conversion information to a marketing or measurement vendor. A vendor may use a JavaScript library, a tracking request or a vendor extension. Select the method required by that vendor.

Adobe Tags (Adobe Launch) manages the events, conditions and actions used to run these tags. Adobe Launch is the previous name and remains the common term used by the Citi implementation team. This guide uses “Launch” for rule configuration.

3 How a Launch rule works

Rule part	Meaning and example
WHEN Event	A page finishes loading, or the application sends a named Direct Call.
IF Condition	The required page, campaign or application values match.
THEN Action	The configured vendor extension action or Custom Code runs.

Example: WHEN the required page loads, IF its pathname and campaign value qualify, THEN run the vendor tag action. The event decides when the rule is evaluated; the conditions decide whether its action should run.

4 Common terms

Term	Simple explanation
Property	Container holding a site's extensions, rules and Data Elements.
Rule	A configured event, any required conditions and one or more actions.
Event	What starts the rule, such as Window Loaded or a Direct Call.
Condition	A check that must qualify before the rule action runs.
Action	Runs Custom Code or a configured vendor action.
Direct Call	A named event triggered with <code>_satellite.track("identifier")</code> .
Data Element	A reusable lookup for a value needed by one or more rules.
Custom Code	JavaScript added to a Launch condition, action or Data Element.
Library	A set of resource revisions built for a selected environment.
Data Layer	Application or analytics values exposed to the tagging implementation.
Vendor Pixel	Vendor tag or request for marketing or measurement.
Conversion Event	A required business event, such as an application outcome.

5 Before you start

Confirm the following details before configuring the rule. Use the agreed business requirement rather than copying another vendor's settings.

Check	Information required
Vendor requirement	Vendor/pixel name, required implementation method and documentation
Business event and journey	Required conversion, page or application journey and when the tag should fire
Event and conditions	Page-load event or application event; required host, path, query and business conditions
Source values	Required application values, globals, Data Elements or query-string values and when they are available
Vendor request	Endpoint, parameter names, exact casing and expected values
Consent	Applicable consent requirements and how the tag follows them
Launch access	Correct property, Development environment, library and implementation access
Testing	Expected event, request, values and navigation behaviour to check

6 Common implementation approach

Define the requirement, select the event, add only the required conditions, identify the source values and configure the vendor action. Save the resources in the appropriate Development library and test the actual journey. Part 3 provides the detailed steps for a new vendor.

Part 2 Vendor 1 BridgeTrack Pixel Implementation

7 BridgeTrack overview

BridgeTrack is used for marketing and conversion tracking in Citi card application journeys. The implementation includes Application Start, Approved, Declined, Pending and Registered events, prequalification events and a separate Funding Start requirement.

The team has completed the migration from Ensignten to Adobe Launch. The existing vendor request structure and conversion values are retained. Ensignten-specific event methods and script insertion are replaced with Launch and native JavaScript methods.

Note: App Start is the main example because it includes required-value checks and retry functionality. Other application events, such as Approved, Declined and Pending, construct and send their BridgeTrack requests directly without App Start's retry function. The NGA Rule identifies the application event and triggers the matching Direct Call.

8 What changed from Ensignten

Implementation	Ensignten	Adobe Launch
App Start event	DOM-loaded event	Core Window Loaded, order 50
Named application event	<code>Bootstrapper.ensEvent.trigger(...)</code>	<code>_satellite.track(...)</code> with the exact Direct Call identifier
Script insertion	<code>Bootstrapper.insertScript(url)</code>	Native JavaScript script creation and <code>appendChild</code>
NGA source values	<code>_dl.pid</code> and <code>_dl.prospect_id</code> , plus rule-specific globals	The same NGA fields and globals are retained
Vendor request	BridgeTrack endpoint and conversion parameters	Endpoint and business parameters retained

DOM-loaded and Window Loaded are different browser events. The selected Launch event is Window Loaded; the App Start checker reads the required values again while retrying.

Named application event example

Ensignten:

```
Bootstrapper.ensEvent.trigger("NGA_application_approved");
```

Adobe Launch:

```
_satellite.track("NGA_application_approved");
```

The matching Launch rule listens for this exact identifier. A rule does not fire merely because its name contains "Approval".

Part A Configure App Start

9 App Start rule setup

A1 Create the Launch rule

In the correct property, create or open the App Start rule. The existing rule is named Third_Party_Bridgetrack_MV_Cards_Conversion_NGAAppStartPages. Keep the App Start event and action together in this rule.

A2 Configure the event

Add Core → Window Loaded. Set the event order to 50. This is the page-load event selected for App Start.

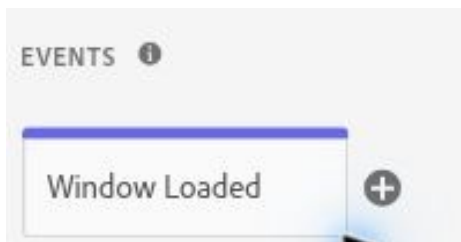


Figure 1. App Start Core Window Loaded event.

A3 Configure the conditions

Condition	Launch configuration
Hostname	Core Custom Code using the existing approved UAT/online hostname condition. Retain that condition rather than inventing a new host list.
Application path	Core Path condition with regular expression enabled: ^(/US/(ag nga)/cards/application/?)\$
Application query	Core Query String condition: parameter app; regular expression ^(UNSOL)\$
Campaign value	Core Value Comparison: %URL_cmp% Starts With afa. This is a separate value check, using the lowercase prefix.

The path condition covers /US/ag/cards/application and /US/nga/cards/application, with an optional trailing slash. The expression is a pattern; it is not a literal application URL.

URL_cmp reads the query parameter cmp. Its parameter-name case setting is separate from the rule's comparison with afa. See Part D for the lookup configuration.

A4 Add the Custom Code action

Add Core → Custom Code and use the existing App Start JavaScript. The action reads the values, checks readiness, builds the BridgeTrack URL and inserts the script.

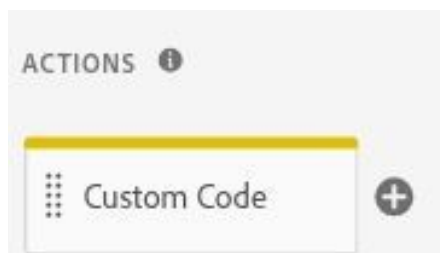


Figure 2. App Start Core Custom Code action.

Note: Understand both the conditions in the Launch interface and any checks inside Custom Code. Other BridgeTrack rules do not automatically use the same conditions.

10 App Start values and retries

A5 Read the required values

The local `_dl` object is read inside `checker()`, so the application values are read again on each check. A missing `window._dl` is handled as an empty object in this action.

```
var _dl = window._dl || {};  
var prodId = _dl.pid || '';  
var prspectId = _dl.prospect_id || '';
```

Value used by App Start	Actual source
prodId	window._dl.pid
prspectId	window._dl.prospect_id
sc	window.sc
businessTypCd	window.businessTypCd
prodType	window.prodType
appType	window.appType
appld	window.appld; included in the URL, outside the six-field readiness check

```
1 var retries = 0;  
2 var maxRetries = 10; // to avoid infinite loop, max 10 seconds  
3  
4 function checker() {  
5   var _dl=window._dl||{};  
6   var prodId=_dl.pid||'';  
7   var prspectId=_dl.prospect_id||'';  
8   var sc = window.sc ||'';  
9   var appId=window.appId||'';  
10  var businessTypCd=window.businessTypCd||'';  
11  var prodType=window.prodType||'';  
12  var appType=window.appType||'';  
13  
14  var allReady = prodId && prspectId && sc && businessTypCd && prodType && appType;  
15  if (allReady) {
```

Figure 3. App Start input values and six-field readiness check.

A6 Retain the readiness and retry function

App Start requires six truthy values: `prodId`, `prspectId`, `sc`, `businessTypCd`, `prodType` and `appType`. The first check runs immediately. If any required value is missing, another check is scheduled after approximately one second.

The action supports up to ten scheduled retries after the first check, giving up to eleven evaluations. Once all six values are present, it constructs the URL and inserts the script. If a value remains missing, it stops after the configured limit and logs a warning. Browser timing can affect the elapsed time.

```
var allReady = prodId && prspectId && sc &&  
  businessTypCd && prodType && appType;
```

Note: `appld` is included in the BridgeTrack URL but is not part of `allReady`. The current action can send an empty `appld` when the six required values are available. Preserve this existing behaviour.

11 BridgeTrack request and script insertion

A7 Retain the vendor request

Vendor host: citi.bridgetrack.com

Vendor path: /track/citicards/s/

App Start conversion: docid=NGA_AppPage

Parameter	Source or meaning
pid	Existing prodId + prodType sequence in the URL expression. Preserve its order and value types.
docid	BridgeTrack conversion identifier; App Start uses NGA_AppPage.
app	appType
appld	Application ID from appld
type	businessTypCd
r	Math.random() * 10000000000000; a random cache-busting value
ProspectID	prspectId; preserve this exact parameter spelling and casing
sc	Rule-specific sc value; App Start reads window.sc

Retain the URL, parameter casing and concatenation. The r value is a cache buster, not a guaranteed unique conversion or business identifier.

A8 Create and insert the script

Enlighten used:

```
Bootstrapper.insertScript(url);
```

Adobe Launch uses:

```
var script = document.createElement("script");
script.type = "text/javascript";
script.async = true;
script.src = url;
(document.head || document.documentElement).appendChild(script);
```

```
18 var script = document.createElement("script");
19 script.type = "text/javascript";
20 script.async = true;
21 script.src = url;
22 (document.head || document.documentElement).appendChild(script);
23 console.log("Pixel fired via recursive checker:", url);
```

Figure 4. Native JavaScript script insertion in the App Start action.

Bootstrapper.insertScript() belongs to Enlighten. Launch uses native JavaScript to create the script, set its type and async flag, assign the URL and append it to the head or document element. This removes the Enlighten dependency.

The Enlighten helper deferred insertion before the first head child; Launch appends immediately. The vendor URL and business parameters remain unchanged.

A9 Save and test

Save the rule and its dependencies in the appropriate Development library, then build Development. Follow Part E to check the event, conditions, values and request. Keep this checker specific to App Start.

Part B Configure Application and Prequalification Events

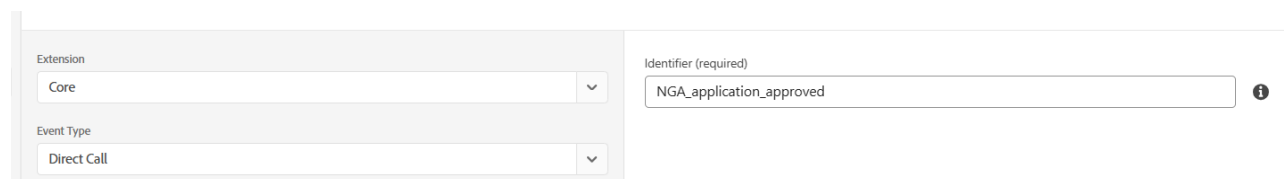
12 Direct Call mappings

Each application or prequalification event uses its own Direct Call and docid. Use the identifier exactly as shown, including capitalization and the existing “pending” and “registration” wording.

Business event	Direct Call identifier	BridgeTrack docid
Approved	NGA_application_approved	NGA_Approved
Declined	NGA_application_declined	NGA_Declined
Pending	NGA_application_pending	NGA_Pending
Registered	NGA_application_registration	NGA_Registered
Prequal Submit	NGA_prequal_submit	NGA_INPQ_SUBMIT
Prequal Approved	NGA_prequal_approved	NGA_INPQ_APPROVE
Prequal Declined	NGA_prequal_declined	NGA_INPQ_DECLINE

B1 Create the matching rule and event

Create or open the rule for the required business event. Add Core → Direct Call and enter the exact identifier. The existing rules use order 50. For Approval, use NGA_application_approved.



The screenshot shows a configuration form for a rule. On the left, there are two dropdown menus: 'Extension' with 'Core' selected and 'Event Type' with 'Direct Call' selected. On the right, there is a text input field labeled 'Identifier (required)' containing the text 'NGA_application_approved'. An information icon (i) is located to the right of the input field.

Figure 5. Core Direct Call configured with NGA_application_approved.

B2 Use the relevant conditions and action

Use the conditions needed by that rule. The existing seven outcome/prequalification rules do not contain outer Launch conditions. Some actions contain their own business checks: the Approval action checks `_dl.site_events` for `app_approved`, `registration_complete` or `application_complete` before building its request.

Add the approved Custom Code for that event. These rules read `_dl.pid` and `_dl.prospect_id` and the required global application values, then build the URL and insert the script directly. They do not include App Start's recursive checker.

B3 Check the conversion and test

Verify the rule's docid against the mapping table. Check the endpoint and all eight parameters, save the rule and its dependencies, build Development and trigger the actual application event. The Direct Call identifier must match; any applicable rule or code checks must also qualify.

13 Funding Start uses a separate page load rule

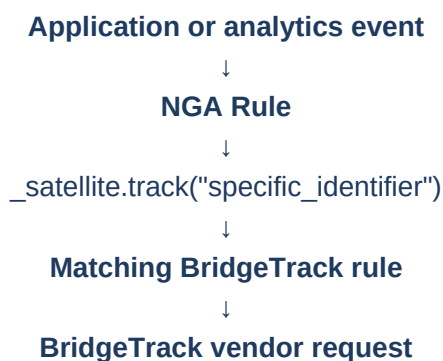
Funding Start follows a separate Window Loaded pattern. Its lookup continues to use window.citiData.pageName. Keep its event, conditions and values separate from the NGA outcome rules.

Funding Start setting	Existing implementation
Launch event	Core Window Loaded, order 50
Hostname	Existing condition for www.citicards.com and sit06, sit02 and sit07.accountonline.com
Path condition	Regular expression: ^(/cards/credit/application/flow\..action/?)\$
Page-name condition	%citidata_pageDef% matches ^(jJRSACQ_Secured_FundAcct)\$
Conversion	docid=dd_fundstart
Request values	window.prodId, window.prspectId, window.applId, window.businessTypCd, window.prodType, window.appType and window.sc
Readiness	Direct URL construction and script insertion; no App Start retry function

Part C Understand the NGA Rule and Direct Calls

14 Keep the event rule and BridgeTrack rule separate

The NGA Rule reads the application values and identifies the required business event. It then triggers the corresponding Direct Call. The matching BridgeTrack Launch rule creates the vendor request.



Application event names

Ensign application event	Launch Direct Call
na_tms_page_view	na_launch_page_view
na_tms_custom_event	na_launch_interaction

The NGA Rule listens for na_launch_page_view and na_launch_interaction. It checks the application state and calls the specific business-event identifier. Dispatching an application Direct Call does not cause every BridgeTrack tag to fire.

```

34  if (isValid) {
35      if (isRegistration) {
36          _satellite.track("NGA_application_registration");
37      } else if (isApproval) {
38          _satellite.track("NGA_application_approved");
39      } /*else if (isPendResolution) {
40          _satellite.track("NGA_application_pend_resolution");
41      }*/ else if (isDeclined) {
42          _satellite.track("NGA_application_declined");
43      } else if (isPended) {
44          _satellite.track("NGA_application_pended");
45      }
46  }
47  // GNADT - 8142 Start of the code

```

Figure 6. NGA Rule code calling the named application Direct Calls.

Approval example

```
_satellite.track("NGA_application_approved");
```

The Approval Launch rule listens for NGA_application_approved and builds docid=NGA_Approved when its code checks qualify. The remaining events use their own identifiers and conversion values from Part B.

Part D Use Data Elements and the Citi Data Layer

15 Reuse the required lookups

A Data Element is a reusable way to read a value needed by one or more rules. The source lookup names below are used by the existing conditions. The descriptive labels explain their purpose; they are not replacements for the existing names.

Source lookup	Purpose	Configuration
URL_cmp	Campaign query value	Core Query String Parameter; parameter name cmp
citidata_pageDef	Funding page name	Core Custom Code reading window.citiData.pageName, with its existing fallback

Campaign lookup and value condition

- Create or open the campaign Data Element using Core → Query String Parameter.
- Set the query parameter name to cmp. The existing lookup is case sensitive for the parameter name.
- Use %URL_cmp% in the App Start Value Comparison condition, with Starts With and afa. This condition checks the value separately.

URL Query String Parameter Name (required)

☐ Allow capitalization differences (case-insensitive)

Figure 7. Campaign Data Element reading query-string parameter cmp.

A default-value setting does not validate a campaign prefix. Check that the lookup returns the expected cmp value and that the separate rule condition qualifies. Keep required storage and fallback settings with the agreed lookup.

Funding page-name lookup

citidata_pageDef reads window.citiData.pageName. When citiData exists but pageName is missing, it returns "na"; when citiData does not exist, it returns undefined. The Funding Start condition compares this lookup with jJRSACQ_Secured_FundAcct.

16 Using the Citi Data Layer

Citi is standardizing the use of `window._dl` for application and analytics values. Before updating a rule that reads `window.citiData`, identify the equivalent `_dl` field and verify its field name, expected value, data type where relevant and availability when the rule runs.

The original NGA Ensign implementation already used `_dl.pid` and `_dl.prospect_id`. Launch retains those mappings; they are not a `citiData`-to-`_dl` change. Funding Start still reads `window.citiData.pageName`. Confirm the equivalent `_dl` mapping before changing that lookup.

Part E Test the BridgeTrack Implementation

17 Check the event and vendor request

E1 Load the correct Development implementation

Use the page and journey that load the required Development library. Confirm that the saved rule, Data Elements and dependencies are included in that library and that its build has completed.

E2 Open Developer Tools

In Chrome or Edge, open Developer Tools and select Network. Enable Preserve log if the journey navigates between pages. Filter for `bridgetrack` or `citi.bridgetrack.com`.

E3 Trigger the required business event

Follow the appropriate page or application journey. For App Start, load a qualifying application path with the required app and cmp query values. For outcomes, trigger the appropriate application event through the journey and check the matching Direct Call.

E4 Inspect the request

Open the BridgeTrack request and inspect its URL and query parameters. Verify the endpoint, docid, pid, app, appld, type, r, ProspectID and sc. Check when the request occurred and compare the values with the application values at that event.

E5 Check missing or duplicate requests

If a request is missing, check the event, conditions and required values. If more than one request appears, check rule execution and the application event count before changing the code. Also check navigation, SPA behaviour and applicable consent behaviour.

A browser request confirms the tag attempted the vendor request. Downstream conversion attribution is checked separately with the relevant media or account team.

18 Testing checklist

Check	What to verify
Event	Correct page-load event or exact Direct Call identifier
Conditions	Required hostname, path, query and internal business checks
Data Elements	Correct source lookup and value returned at the event
Application values	Correct <code>_dl</code> fields and global values for the specific rule
Request	<code>citi.bridgetrack.com/track/citicards/s/</code> and correct parameter casing
Conversion values	Correct docid and all business values, including sc
App Start	Six-field readiness; immediate check and retries when needed; appld behaviour retained

Check	What to verify
Event count	Required tag fires at the correct business event; investigate unintended repeated requests
Navigation and consent	SPA/full-page navigation and applicable consent behaviour

Part 3 Common Support and Future Implementation

19 Troubleshooting

Problem	Possible reason	What to check
Pixel does not fire	Wrong library, event or action	Confirm the Development build, rule event and required action.
Conditions do not qualify	Host, path, query or business value differs	Check the actual page values, regex setting and code checks.
Approved pixel does not fire	Direct Call identifier does not match	Compare the NGA Rule identifier with the Approval rule event.
Incorrect docid	Action copied from a different event	Compare the action value with the Part B mapping table.
Missing source values	Wrong field or value not available	Check the specific _dl/global field or Data Element at the event.
App Start reaches its retry limit	One of six required values remains missing	Check prodId, prspectId, sc, businessTypCd, prodType and appType.
Duplicate vendor requests	Repeated application events or multiple matching rules	Compare rule execution, Direct Calls and Network initiators.
Campaign condition fails	cmp key or afa comparison does not match	Check URL_cmp, query-name case and the separate Starts With condition.
Development changes not visible	Required revision or build is missing	Check the library resources, build result and page embed.
Old Ensign method is called	Executed code still uses Bootstrapper	Replace the required method with Launch or native JavaScript for that vendor.

20 Implement a new third party pixel

Use this process for a new marketing or conversion requirement. BridgeTrack is one vendor example; select the new vendor's required method and values.

1 Understand the requirement

Identify the vendor, business event, page or journey, firing point, endpoint and required parameters.

2 Select the event

Choose Window Loaded or another suitable page-load event for page requirements, Direct Call for a named application/business event, or another appropriate Launch event.

3 Define the conditions

Use the hostname, path, query parameter, Data Element or application-state checks required by this requirement.

4 Identify the source values

Map each vendor value to the correct _dl field, global, Data Element, query parameter or application value. Verify the business meaning before reusing a source.

5 Check value availability

Use values directly when available. Add a suitable readiness check only when a required value arrives later. Do not add App Start retries to every tag.

6 Build the vendor request

Retain the vendor-defined endpoint, parameter names, casing, values and request structure. Use that vendor's parameters rather than BridgeTrack's.

7 Select the implementation method

Use an approved vendor extension where suitable, or the required Custom Code/native JavaScript. Native script creation applies when the vendor requires a script. Use Direct Calls where the application event requires them.

8 Test the implementation

Check the correct rule, event, conditions, values, request and firing time. Check unintended duplicate requests, navigation/SPA behaviour and applicable consent.

9 Document the implementation

Record the configuration and test result using the template in Section 22. Keep enough detail for another team member to maintain the tag.

Note: If one rule identifies several application states, keep that event rule separate from the matching vendor tag rules where required. Use the exact Direct Call identifiers between them.

21 Quick implementation reference

Requirement	Suggested Launch implementation
Page-load tag	Appropriate page-load event
Named application or business event	Direct Call with the exact identifier
Reusable page, query or application value	Data Element
Required vendor JavaScript	Custom Code/native script creation where the vendor requires it
Vendor supports a suitable extension	Approved vendor extension and its native actions
Required value becomes available later	Readiness/retry check only where required

22 Document the implementation

Copy and complete this record for each newly implemented pixel. Use the actual configuration names and values.

Field	Implementation details
Vendor	[Enter vendor/pixel]
Business requirement	[Enter event/conversion and page or journey]
Adobe Tags rule name	[Enter exact rule name]
Event type	[Enter selected Launch event]
Direct Call identifier	[Enter exact identifier, if applicable]
Conditions	[Enter interface conditions and internal code checks]
Source values	[Enter fields, globals/query values and their availability]
Data Elements	[Enter exact lookup names and configurations]
Vendor endpoint	[Enter host/path or library URL]
Vendor parameters	[Enter names, casing and value mappings]
Testing result	[Enter journey, expected/observed behaviour and result]
Implementation notes	[Enter relevant rule-specific details]

23 Key implementation principles

- Select the event based on the actual business requirement.
- Retain the vendor request structure when the requirement is unchanged.
- Use Launch or native JavaScript instead of executed Ensign-specific methods.
- Use the exact Direct Call identifier and the required rule conditions.
- Identify the actual source of every required value and when it becomes available.
- Keep the event rule and tag rule separate when the implementation requires it.
- Reuse an existing pattern only when it matches the new requirement.
- Keep the implementation simple and maintainable for the Citi team.

24 Team contacts

Business requirement / P1T contact: [Add team or contact]

Adobe Tags implementation contact: [Add team or contact]

Analytics and validation contact: [Add team or contact]